ReactJS Technical Assessment

Dog Breed Viewer

Instructions

Project Overview

The goal of this challenge is to build a simple single-page application using React that allows users to browse a list of dog breeds and view random images for a selected breed.

This test should take **approximately 3-6 hours** to complete but feel free to take as long as you need as different approaches will require varying implementation effort.

Core Requirements

Your application must meet the following criteria:

1. **Fetch and Display Breed List:**
    - When the application first loads, it should fetch a list of all dog breeds from the public API.

cloudsmiths
www.cloudsmiths.co.za
© 2025 CloudSmiths. All rights reserved.

CloudSmiths (Pty) Ltd
Reg Number: 2014/143642/07
VAT: 4380269391
Director: Jason Timm

JHB - Dynamic Technologies House, 2 Venue Street, Melrose Estate

- Display this list of breeds to the user, for example, in a dropdown menu or a scrollable list.
- The endpoint returning all breeds described below contains sub-breeds for some of the top level breeds. For example the “terrier” breed contains several sub-breeds whereas the “beagle” breed does not. **You only need to store and display the parent breeds.**

2. **Select a Breed and View Images:**
   - When a user selects a specific breed from the list, the application must fetch **3** random images of that breed.
   - Display these images clearly on the page.
   - The images may be displayed on a new screen, modal or some other container on the page as long as the design is unobtrusive and allows for easy use and navigation.
3. **Add a search/filter input field to easily find a specific breed in the list.**
4. **Handle API Errors:**
   - If an API call fails for any reason, display a user-friendly error message.
5. **Dynamic Updates:**
   - The images displayed should automatically update whenever the user selects a different breed from the list.
6. **Loading States:**
   - Display a loading indicator (e.g., text like "Loading...") while the initial breed list is being fetched.
   - Display a separate loading indicator while the images for a selected breed are being fetched.

API Endpoints

Please use the free and open Dog CEO API. No API key is required.

cloudsmiths www.cloudsmiths.co.za
© 2025 CloudSmiths. All rights reserved.

Investigate the API documentation above to find the relevant endpoints to do the following:

- **List all breeds**
- **Get random images for a selected breed**

Technical Specifications

- The application must be built using **React (You may use Vite or a similar tool to bootstrap your React app)**.
- You must use **functional components** and either **React Hooks** or **Zustand** for state management.
- You do not need to use any external UI component libraries (like Material-UI). However any UI Component library of your choosing is acceptable. It is also acceptable to use UI templates provided by these libraries provided they fit the use case and you edit them accordingly.

Evaluation Criteria

We will be evaluating your submission based on the following:

- **Functionality:** Does the application meet all of the core requirements?
- **Code Quality:** Is the code clean, well-organized, and easy to read? Are components logically structured?
- **React Fundamentals:** Is state managed correctly? Are side effects (API calls) handled properly?
- **User Experience (UX):** Are the loading and error states handled gracefully, providing clear feedback to the user?

Bonus Points (Optional)

© 2025 CloudSmiths. All rights reserved.

If you have extra time and want to showcase more of your skills, consider adding one or more of the following features:

- **Rate Limits:** Handle rate limiting errors appropriately with the relevant retry logic.
- **Cache:** Make use of a local cache mechanism of your choice to store appropriate responses from the API.
- **Unit Testing:** Write a simple unit test for one of your components or a utility function. (Using a library like **Jest**/**React Testing Library** is recommended).
- **User Auth:** Create a log in screen and use the free API documented here https://dummyjson.com/docs/auth to authenticate a user when they sign into your app and display their details somewhere on the screen. Ensure you follow best practices for storing their credentials and handling expired tokens.

Optional Backend Challenge (Node.js or Python)

For candidates who wish to showcase their full-stack capabilities, this optional challenge extends the project with a simple Node.js backend to manage "favourite" images.

Backend Requirements (Node.js/Python)

1. **Create a Server:** Set up a simple backend server using Node.js + Express/NestJS **OR** Flask/FastAPI/Django.
2. **Implement API Endpoints:** Create the following endpoints to manage a list of favourite dog images:
    - `GET /api/favourites`: Returns a list of all favourited image URLs.
    - `POST /api/favourites`: Adds a new image URL to the favourites list. The request body should be `{ "imageUrl": "..." }`.
    - `DELETE /api/favourites`: Removes an image URL from the favourites list. The request body should be `{ "imageUrl": "..." }`.

© 2025 CloudSmiths. All rights reserved.

3. **Data Persistence:** For this exercise, you may but do **not** need to set up a database. You can persist the data by simply reading from and writing to a local `favourites.json` file on the server. Alternatively you may make use of a SQLite or similar local database feature.

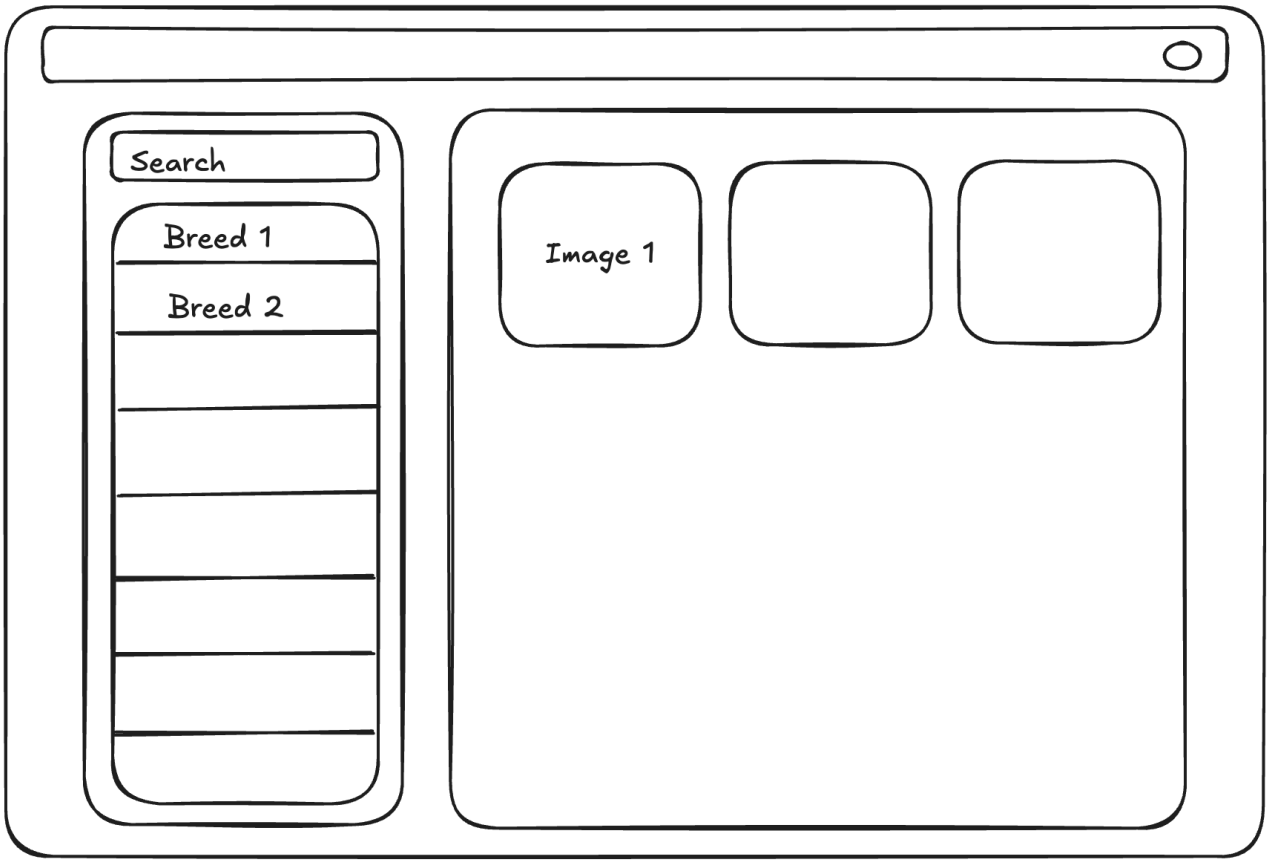
Frontend Requirements (React)

1. **"Favourite" Functionality:** Add a "Favourite" button (e.g., a star icon) to each dog image displayed. Clicking this button should call your `POST /api/favourites` endpoint.
2. **Display Favourites:** Add a "My Favourites" section or page to the application. This section should fetch the list of favourite images from your `GET /api/favourites` endpoint and display them.
3. **"Un-Favourite" Functionality:** In the "My Favourites" view, allow a user to remove an image, which should call your `DELETE /api/favourites` endpoint.

© 2025 CloudSmiths. All rights reserved.

Design Hint

Feel free to come up with your own design and layout, however should you prefer to work from a basic UX design you may use the following as reference.



Submission

Please provide us with a link to a public Git repository (e.g., GitHub, GitLab) or a .zip file containing your source code. The repository should include a `README.md` file with clear instructions on how to install dependencies and run the application locally.

© 2025 CloudSmiths. All rights reserved.

© 2025 CloudSmiths. All rights reserved.